Academic Copilot

Read research papers like a professor explains them.

Live Demo | GitHub Link | Authors: Garima Jain, Akriti Agarwal | Columbia University

1. The User

Academic Copilot serves three overlapping groups united by the same frustration: research papers are hard to read without guidance, and no expert is available on demand.

Graduate students preparing for quals or reading outside their subfield need structured comprehension at 11 pm — not just a summary, but an understanding of what the paper claims, whether to trust it, and how it fits the broader conversation. **Researchers doing literature review** need to rapidly triage 20–50 papers: what gap does each address, what are its limitations, which ones merit a full read. **Professionals and undergraduates** — a data scientist trying to understand a paper her team cited, a student working through dense course material, a junior researcher new to a field — need the same guided comprehension but lack any institutional support. The pipeline extends naturally to textbook chapters, lecture notes, and syllabi, broadening the addressable market to every university student.

> **Common thread:** time-constrained users trying to extract structured understanding from dense technical text, without a subject-matter expert on call.

2. The Problem

Users today rely on fragmented workarounds: skimming abstracts, pasting sections into a general-purpose chatbot with no persistent context, reading blog summaries that are often incomplete, or waiting for a labmate to respond. General LLM chat requires the user to know what to ask and provides no structure, no field context, no critique, and no way to test comprehension. Semantic Scholar surfaces related work but explains nothing. No existing tool combines structured breakdown, contextual positioning, peer-review-style critique, and interactive tutoring grounded in the specific paper the user has in hand.

Academic Copilot closes that gap. In under one minute, any PDF or arXiv link becomes a professor-guided reading session: a five-section structured breakdown with difficulty ratings, a synthesized field summary with related work, a critique surfacing what the paper does *not* prove, a conversational tutor the user can interrogate at any depth, and a downloadable study guide.

3. The Economics

Pricing and unit economics

Freemium: 5 free analyses/month; $12/month Pro unlimited. Each analysis costs ∼25,000 tokens across 4 agents (∼500k tokens/user/month). At Groq paid rates ($0.59/M input, $0.79/M output) this is **$0.35/user/month**, yielding a **97% gross margin** and $11.65 net per Pro user.

At Claude Sonnet rates the cost rises to ∼$3.00, giving a 75% margin. At 10,000 Pro users: **$120k MRR, ∼$116k gross profit**.

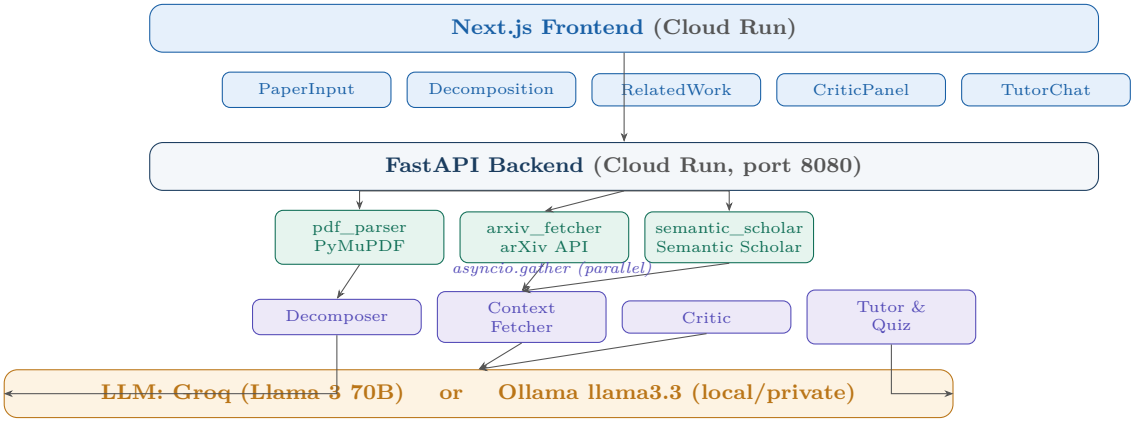
Where it breaks

At 100+ analyses/month on Sonnet-tier pricing, cost reaches ∼$15/user, wiping out margin. Mitigations: strict free-tier rate limits; Groq or Ollama as default backends; a higher-priced Power tier for heavy users. The most important cost control is already in the architecture: abstract-first PDF parsing (first 4 pages only) cuts token usage 60–70% versus full-paper ingestion, reducing the per-analysis cost before any pricing decision is made.

4. Why These Technical Choices?

Each major architectural decision maps directly to a user need, product requirement, or economic constraint.

System Architecture



Multi-agent fan-out

Four focused agents — Decomposer, Context Fetcher, Critic, Tutor/Quiz — each receive a tight task-specific prompt and return a typed Pydantic model. Running Context and Critic in parallel via `asyncio.gather` after the Decomposer finishes halves latency versus sequential execution, keeping the under-one-minute promise to the user.

Structured outputs and the notes artifact

Every agent returns a typed Pydantic model, not free-form text. This lets the frontend render each section deterministically and enables `_build_notes_markdown()` to stitch all agent outputs into a coherent downloadable study guide. Without typed outputs both the UI and the artifact would require fragile text parsing.

Abstract-first parsing and dual LLM backend

Reading only the first 4 pages reduces average input from ∼60,000 tokens to ∼4,000, cutting per-analysis cost by 60–70% — the single most impactful economic decision in the system. Supporting both Groq and Ollama serves individual users and privacy-sensitive institutions respectively, and future-proofs the stack against provider pricing changes.

In-memory cache and tool calling

The paper cache (`_paper_cache`, keyed by `md5(title)[:12]`) drops the marginal cost of a tutor message from ∼$0.06 to ∼$0.001 by avoiding full re-analysis on each `/ask` call. The Context Fetcher uses `@tool_plain` to call Semantic Scholar live on every analysis — no vector database, no stale embeddings, zero storage cost.

5. Class Concepts in Practice

Concept	File	Implementation
Multi-Agent Patterns	`orchestrator.py`	Four agents with `asyncio.gather` fan-out; Decomposer feeds parallel Context + Critic workers
Tool Calling	`context_fetcher.py`	`@context_agent.tool_plain` registers `find_related_papers`; LLM decides when to invoke it against Semantic Scholar API
Chatbots	`tutor.py`	`/ask` injects last 6 messages of history per request; "Explain simpler" chip enables iterative refinement
Agents as Functions	`agents/*.py`	`decompose_paper()`, `fetch_context()`, `critique_paper()`, `ask_tutor()` — typed async functions returning Pydantic models; composable and independently testable